# MercaPOS · Informe de metrología de software

Métricas de producto, proceso y recolección sistemática de datos

PPI115 GT02 · Fase 4 · 13 de noviembre de 2026
Entorno: AMD Ryzen 5 7520U, 8 hilos lógicos, 7 GiB RAM · Linux 6.14 · CouchDB 3.3 en Docker
Backend Rust/Axum (binario debug) · Frontend Flutter Web

## 1. Resumen ejecutivo

Se amplió la instrumentación del backend Rust con histogramas Prometheus de latencia por endpoint, medidores de hilos y memoria residente del proceso. Se crearon dos scripts reproducibles: uno muestrea CPU, RAM e hilos de la API y de CouchDB; el otro genera carga concurrente, mide latencias por transacción y captura las métricas antes y después. La corrida documentada envió 200 compras concurrentes (30 hilos) sin errores: 33.00 operaciones/s, latencia p50 de 884.53 ms y p99 de 969.52 ms. La API promedió 9.05 % de CPU y 44.10 MiB; CouchDB alcanzó 229.91 % de CPU por su trabajo multihilo. El análisis estático y las compilaciones fueron estables.

| 200 / 200 | 33.00 | 969.52 ms | 0 |
|---|---|---|---|
| TRANSACCIONES ACEPTADAS | OPERACIONES POR SEGUNDO | LATENCIA P99 CLIENTE | ERRORES HTTP 5XX |

## 2. Objetivos y correspondencia con ISO/IEC 25010

| Categoría | Requisito del proyecto | Implementación / evidencia |
|---|---|---|
| Producto | Consumo de CPU y RAM del binario Rust y de CouchDB | Script `muestrear_recursos.py` con muestreo de `/proc` y `docker stats`; medidores `mercapos_process_threads` y `mercapos_process_resident_memory_bytes`. |
| Producto | Mantenibilidad estática del backend y del frontend | `cargo fmt --check`, `cargo clippy -D warnings` y `flutter analyze`, todos sin hallazgos. |
| Proceso | Tiempos de ciclo y estabilidad de compilación | Medición de comandos de compilación/pruebas y tres compilaciones consecutivas exitosas. |
| Datos | Latencia por endpoint, hilos y tiempo de transacción bajo concurrencia | Histograma `mercapos_http_request_duration_milliseconds`, gauge de hilos y `prueba_carga.py` con 200 transacciones concurrentes. |

ISO/IEC 25010 se usa como marco: eficiencia de desempeño (tiempo de respuesta y utilización de recursos), mantenibilidad (análisis estático) y fiabilidad observada (ausencia de errores 5xx durante la carga).

## 3. Instrumentación añadida al backend

En `mercapos_api/src/presentation/telemetry.rs` se agregó un histograma acumulado por método y ruta normalizada (sin IDs, para evitar cardinalidad ilimitada), además de medidores del proceso leídos de

`/proc/self/status`:

| Métrica Prometheus | Tipo | Significado |
|---|---|---|
| `mercapos_http_request_duration_milliseconds` | histograma | Distribución de duración HTTP por método y ruta; buckets de 1 a 5000 ms más `+Inf`. |
| `mercapos_http_requests_total` / `..._errors_total` | contador | Solicitudes totales y respuestas 5xx. |
| `mercapos_http_route_requests_total` | contador | Conteo por método, ruta normalizada y estado. |
| `mercapos_process_threads` | gauge | Hilos reales del proceso Rust en el momento de la consulta. |
| `mercapos_process_resident_memory_bytes` | gauge | Memoria residente (RSS) del proceso Rust. |

`GET /metrics` sigue restringido a una sesión con rol administrador. La duración registrada incluye el manejador HTTP y la persistencia en CouchDB; no se etiqueta con IDs de operación, usuario o documento.

## 4. Scripts de recolección

### 4.1 `mercapos_api/scripts/muestrear_recursos.py`

Muestrea periódicamente el proceso Rust y el contenedor de CouchDB. Para la API lee `/proc/<pid>/status` (RSS e hilos) y `/proc/<pid>/stat` (ticks de CPU) y calcula el porcentaje entre muestras. Para CouchDB ejecuta `docker stats --no-stream`. Genera un CSV por muestra y un LOG con mínimos, medias y máximos.

```
python3 scripts/muestrear_recursos.py \
  --api-container mercapos-api-1 \
  --couchdb-container mercapos-couchdb-1 \
  --duracion 40 --intervalo 1 --salida resultados_metrologia
```

Parámetros: `--api-pid` o `--api-container`; `--couchdb-container` (autodetectado); `--duracion`; `--intervalo`; `--salida`. En ejecución local se usa `--api-pid`; en Docker, `--api-container`. La CPU puede superar 100 % cuando el proceso usa varios hilos.

### 4.2 `mercapos_api/scripts/prueba_carga.py`

Inicia sesión, obtiene un producto, captura `/metrics`, envía N compras concurrentes por `POST /operaciones` con identificadores únicos, mide la latencia de cada transacción y vuelve a capturar `/metrics`. Calcula throughput, p50/p95/p99 y el delta de la telemetría del servidor (conteo, suma de duración y promedio por transacción).

```
python3 scripts/prueba_carga.py \
  --api http://127.0.0.1:3000 \
  --usuario usuario:demo-admin --password MercaPOS2026 \
  --puesto puesto:demo \
  --operaciones 200 --concurrencia 30 \
  --salida resultados_metrologia
```

Genera: CSV por transacción, LOG resumen, y las exposiciones Prometheus `metricas_antes_*.txt` y `metricas_despues_*.txt`. La latencia del cliente incluye red local, HTTP y persistencia; el promedio del servidor se obtiene del histograma Prometheus.

## 5. Metodología de la corrida

- Base aislada `mercapos_medicion` y autenticación `mercapos_medicion_auth`, con esquema v2 instalado y seis productos de prueba.
- Segunda instancia de la API en `127.0.0.1:3001` (mismo binario debug) para no alterar la base de demostración.
- Carga: 200 compras, 30 solicitudes concurrentes, un renglón por compra.
- Muestreo de recursos: 40 s, intervalo nominal de 1 s (21 muestras efectivas; `docker stats` añade latencia).
- Fecha: 13/11/2026, 13:37–13:39 (hora local, UTC−06:00).
- Cliente, API y CouchDB en el mismo equipo; CouchDB dentro de Docker.

## 6. Resultados de carga concurrente

| Métrica | Valor |
|---|---|
| Operaciones enviadas / aceptadas | 200 / 200 (100 %) |
| Errores HTTP o 5xx | 0 |
| Duración total de la carga | 6.061 s |
| Throughput | 33.00 operaciones/s |
| Latencia mínima / media | 103.71 ms / 841.32 ms |
| Latencia p50 / p95 / p99 | 884.53 ms / 922.88 ms / 969.52 ms |
| Latencia máxima | 1003.89 ms |
| Telemetría servidor: solicitudes / tiempo acumulado | 200 / 167 508 ms |
| Tiempo medio por transacción (servidor) | 837.54 ms |
| Hilos / RSS del proceso (después) | 10 hilos / 44.30 MiB |

La diferencia entre la media del cliente (841.32 ms) y la del servidor (837.54 ms) es de aproximadamente 3.8 ms, atribuible al trayecto HTTP local. El histograma del servidor coincide con las latencias del cliente y no registró valores fuera del rango observado.

### 6.1 Distribución de latencia de esta corrida




Conteo no acumulado de las 200 transacciones; se calculó a partir del CSV generado por el script.

## 7. Consumo de recursos

| Componente | Métrica | Mínimo | Media | Máximo |
|---|---|---|---|---|
| API Rust (PID 6046) | CPU (%) | 0.00 | 9.05 | 50.90 |
| | RSS (MiB) | 43.24 | 44.10 | 44.30 |
| | Hilos | 9 | 9.24 | 10 |
| CouchDB 3.3 (Docker) | CPU (%) | 0.41 | 34.84 | 229.91 |
| | Memoria (MiB) | 89.54 | 93.01 | 94.49 |

El pico de CPU de CouchDB supera 100 % porque Docker suma el uso de varios hilos; la API se mantuvo por debajo de un núcleo completo (50.90 %). La memoria de la API fue estable durante la carga (variación menor a 1.1 MiB) y CouchDB mantuvo un rango estrecho. El muestreo incluye segundos previos y posteriores a la carga, por lo que los mínimos reflejan el estado en reposo.

> El binario medido fue la compilación **debug** para conservar paridad con el entorno de desarrollo. Una compilación `--release` puede reducir CPU y latencia; la comparación entre perfiles queda como siguiente paso, no como resultado de este informe.

## 8. Calidad estática y métricas de proceso

| Comando | Resultado | Tiempo |
|---|---|---|
| `cargo fmt --check` | **OK** | 0.41 s |
| `cargo clippy --all-targets --locked -- -D warnings` | **OK, sin advertencias** | 0.62 s |
| `cargo test --locked` | **32 pruebas aprobadas; 3 integraciones ignoradas por diseño** | 2.52 s |
| `cargo build --locked` ×3 | **3/3 compilaciones exitosas** | 0.12 / 0.12 / 0.13 s |
| `flutter analyze` | **No issues found** | 8.88 s |
| `flutter test --platform chrome` | **8 pruebas aprobadas** | 47.03 s |
| `flutter build web --release` | **Compilación correcta** | 52.53 s |

Los tiempos de Rust corresponden a compilaciones incrementales con caché; no representan una compilación limpia. La estabilidad se evaluó con tres compilaciones consecutivas sin fallos. El ciclo completo de verificación del backend (formato, Clippy y pruebas) tomó 3.55 s en este equipo.

## 9. Limitaciones

- Las mediciones corresponden a un equipo local y a una sola corrida; no constituyen una muestra estadísticamente representativa.
- Cliente, API y CouchDB comparten CPU y memoria; el rendimiento observado no equivale a un despliegue en red real.

- No se midieron pérdida de paquetes, ancho de banda ni latencia WAN. No se evaluó restauración desde respaldo.
- El tiempo de negocio se aproxima con la duración del manejador HTTP; no se separan subetapas internas de CouchDB.
- La recolección de recursos depende de `/proc` y `docker stats`; en otros sistemas operativos debe adaptarse.

## 10. Conclusiones

La instrumentación cubre los tres bloques solicitados: eficiencia de desempeño con histogramas por endpoint y medidores de recursos, mantenibilidad con análisis estático reproducible, y datos sistemáticos de latencia y concurrencia mediante dos scripts versionados. La carga de 200 transacciones concurrentes finalizó sin rechazos ni errores 5xx, con una distribución de latencia concentrada entre 500 y 1000 ms y un consumo de memoria estable. Para la fase de análisis estadístico se recomienda repetir la corrida con más muestras, comparar perfiles debug/release y medir en un equipo o red distintos.

### Reproducción

```
# 1) Muestreo de recursos mientras corre la carga (dos terminales)
python3 mercapos_api/scripts/muestrear_recursos.py \
  --api-pid $(pgrep -f target/debug/backend) \
  --couchdb-container mercapos-couchdb \
  --duracion 40 --salida resultados_metrologia

# 2) Carga concurrente
python3 mercapos_api/scripts/prueba_carga.py \
  --api http://127.0.0.1:3000 \
  --usuario usuario:demo-admin --password MercaPOS2026 \
  --puesto puesto:demo --operaciones 200 --concurrencia 30 \
  --salida resultados_metrologia
```

---

Archivos fuente de este informe: `resultados_metrologia/carga_20261113_133815.log`, `resultados_metrologia/recursos_20261113_133812.log`, `resultados_metrologia/proceso_y_calidad.log`, `resultados_metrologia/metricas_despues_20261113_133815.txt`. MercaPOS · PPI115 GT02 · 2026.